AI LAB ASSIGNMENT 11

NAME:Burra Tejaswini

ROLL_NO:2403A54095

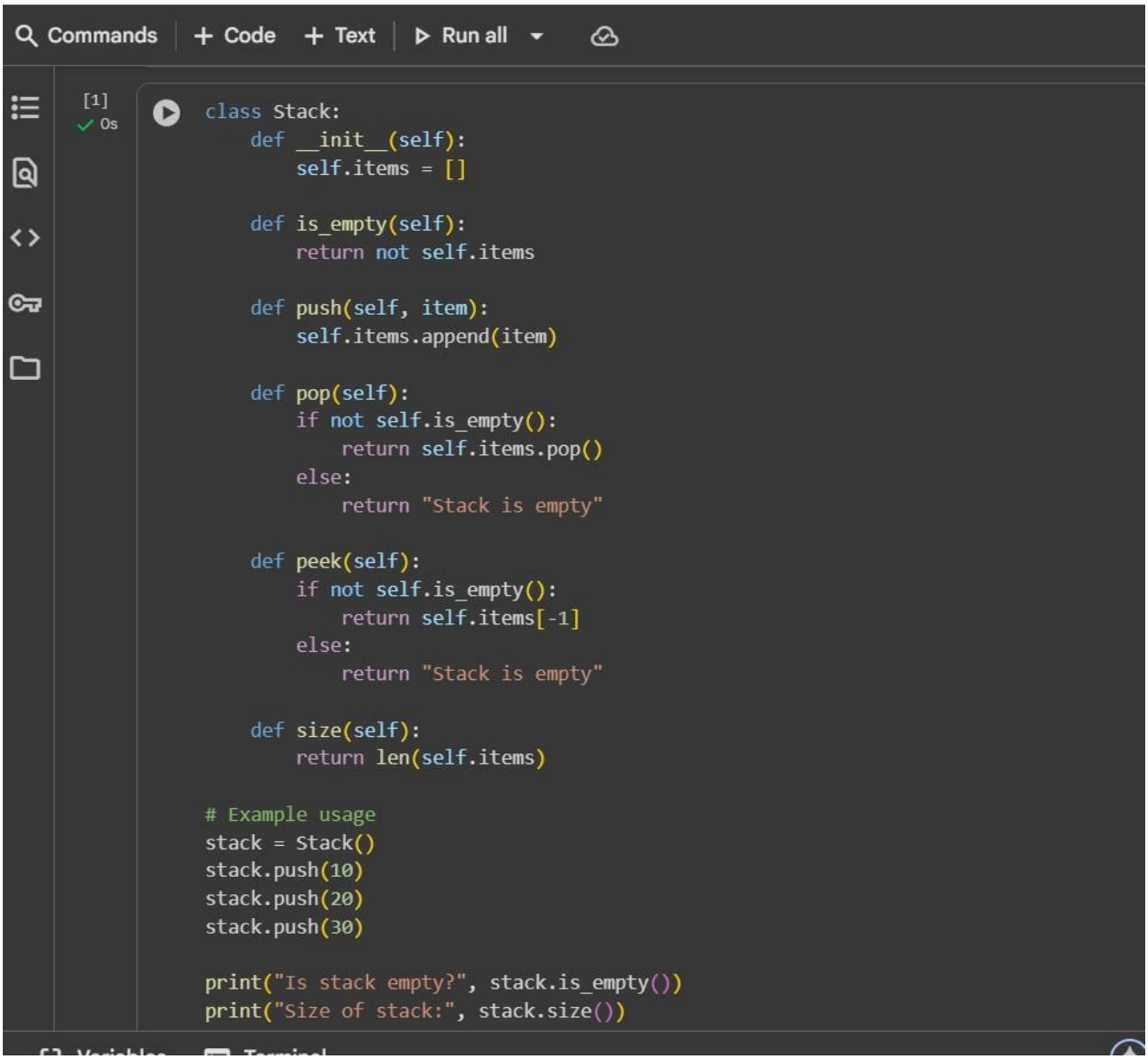
TASK 1:

Task 1: Implementing a Stack (LIFO)
• Task: Use AI to help implement a Stack class in Python with the
following operations: push(), pop(), peek(), and is_empty().
• Instructions:
o Ask AI to generate code skeleton with docstrings.
o Test stack operations using sample data.
o Request AI to suggest optimizations or alternative implementations (e.g., using collections.deque).
• Expected Output:
o A working Stack class with proper methods, Google-style docstrings, and inline comments for tricky parts.
e optimized with

PROMPT :

WRITE A PROGRAM IN PYTHON TO GENERATE A CLASSFOR STACK USING

LIST



GENERATE A CODE FOR TESTING THE STACK

```python
# Create a new stack instance
my_stack = Stack()

# Test push operation
print("Pushing elements onto the stack:")
my_stack.push("Apple")
my_stack.push("Banana")
my_stack.push("Cherry")
print("Stack after pushes:", my_stack.items)
print("Size after pushes:", my_stack.size())

# Test peek operation
print("\nPeeking at the top element:", my_stack.peek())

# Test pop operation
print("\nPopping elements from the stack:")
print("Popped:", my_stack.pop())
print("Popped:", my_stack.pop())
print("Stack after pops:", my_stack.items)
print("Size after pops:", my_stack.size())

# Test is_empty operation
print("\nIs stack empty?", my_stack.is_empty())

# Pop remaining element
print("Popped:", my_stack.pop())
print("Stack after final pop:", my_stack.items)
print("Size after final pop:", my_stack.size())

# Test is_empty again
print("Is stack empty?", my_stack.is_empty())

# Test popping from an empty stack
print("\nTrying to pop from an empty stack:", my_stack.pop())
```

```
print("\nTrying to pop from an empty stack:", my_stack.pop())
```

```
Pushing elements onto the stack:
Stack after pushes: ['Apple', 'Banana', 'Cherry']
Size after pushes: 3

Peeking at the top element: Cherry

Popping elements from the stack:
Popped: Cherry
Popped: Banana
Stack after pops: ['Apple']
Size after pops: 1

Is stack empty? False
Popped: Apple
Stack after final pop: []
Size after final pop: 0
Is stack empty? True

Trying to pop from an empty stack: Stack is empty
```

GIVE THE OPTIMATIONS AND ALTERNATIVES USING COLLECTION AND DEQUE

```python
from collections import deque

# Create a deque instance to use as a stack
deque_stack = deque()

# Push elements onto the deque (stack)
deque_stack.append(10)
deque_stack.append(20)
deque_stack.append(30)

print("Deque stack after pushes:", deque_stack)

# Peek at the top element (last element added)
if deque_stack:
    print("Top element (peek):", deque_stack[-1])
else:
    print("Deque stack is empty")

# Pop elements from the deque (stack)
print("Popped element:", deque_stack.pop())
print("Popped element:", deque_stack.pop())

print("Deque stack after pops:", deque_stack)

# Check if the deque stack is empty
print("Is deque stack empty?", not deque_stack)

# Try popping from an empty deque stack
try:
    print("Trying to pop from an empty deque stack:", deque_stack.pop())
except IndexError:
    print("Cannot pop from an empty deque stack (IndexError)")
```

```
Deque stack after pushes: deque([10, 20, 30])
Top element (peek): 30
Popped element: 30
Popped element: 20
Deque stack after pops: deque([10])
Is deque stack empty? False
Trying to pop from an empty deque stack: 10
```

TASK 2:

Queue Implementation with Performance Review

• Task: Implement a Queue with enqueue(), dequeue(), and is_empty()

methods.

• Instructions:

o First, implement using Python lists.

o Then, ask AI to review performance and suggest a more

efficient implementation (using collections.deque).

• Expected Output:

o Two versions of a queue: one with lists and one optimized with

deque, plus an AI-generated performance comparison.

PROMPT :

WRITE A PYTHON QUEUE CLASS USING A LIST .ADD METHODS LIKE

ENQUEUE , DEQUEUE , IS_EMPTY

```python
class Queue:
    def __init__(self):
        self.items = []

    def is_empty(self):
        return not self.items

    def enqueue(self, item):
        self.items.append(item)

    def dequeue(self):
        if not self.is_empty():
            # Removing from the front of the list (index 0)
            return self.items.pop(0)
        else:
            return "Queue is empty"

    def size(self):
        return len(self.items)

    def peek(self):
        if not self.is_empty():
            return self.items[0]
        else:
            return "Queue is empty"

# Example Usage
queue = Queue()
queue.enqueue(10)
queue.enqueue(20)
queue.enqueue(30)
```

```
# Example usage
queue = Queue()
queue.enqueue(10)
queue.enqueue(20)
queue.enqueue(30)

print("Is queue empty?", queue.is_empty())
print("Size of queue:", queue.size())
print("Front element:", queue.peek())
print("Dequeued element:", queue.dequeue())
print("Size after dequeue:", queue.size())
print("Dequeued element:", queue.dequeue())
print("Dequeued element:", queue.dequeue())
print("Dequeued element:", queue.dequeue()) # Trying to dequeue from an empty queue
```

```
Is queue empty? False
Size of queue: 3
Front element: 10
Dequeued element: 10
Size after dequeue: 2
Dequeued element: 20
Dequeued element: 30
Dequeued element: Queue is empty
```

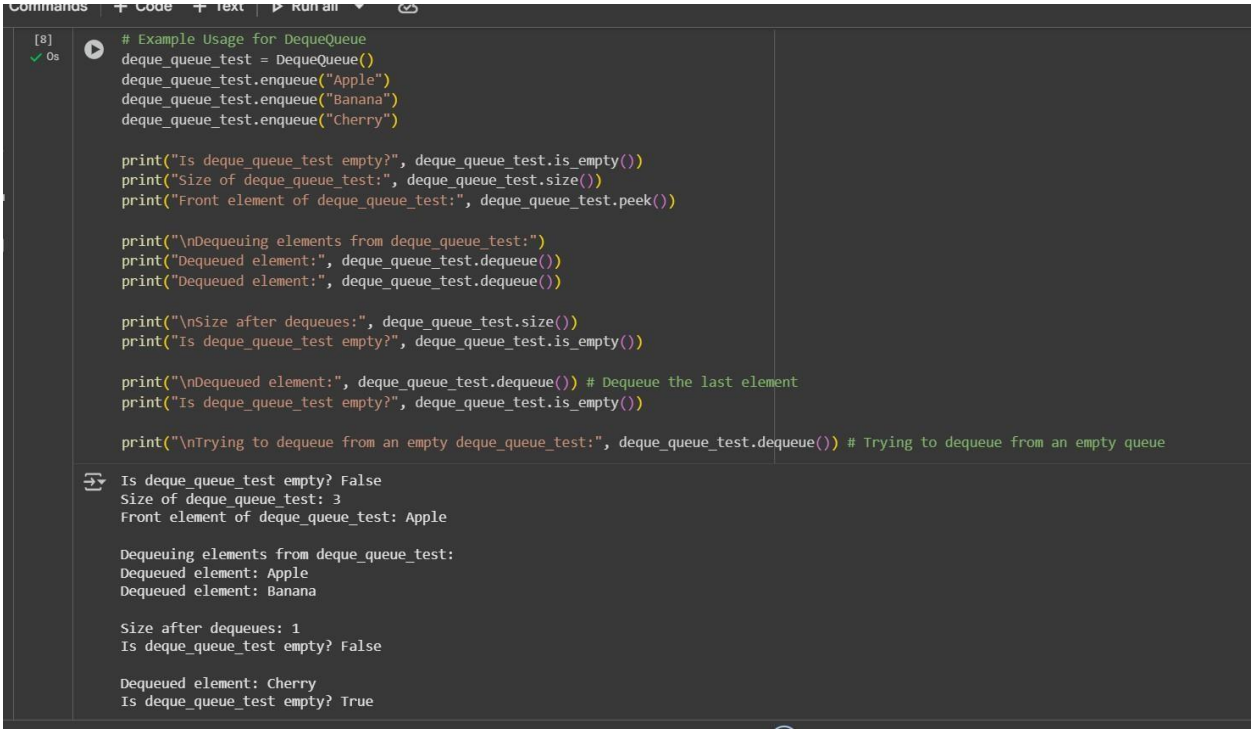
REWRITE THE QUEUE CLASS USING COLLECTIONS , DEQUEUE FOR EFFICIENCY

```python
from collections import deque

class DequeQueue:
    def __init__(self):
        self.items = deque()

    def is_empty(self):
        return not self.items

    def enqueue(self, item):
        self.items.append(item) # Append to the right (end of the queue)

    def dequeue(self):
        if not self.is_empty():
            return self.items.popleft() # Pop from the left (front of the queue)
        else:
            return "Queue is empty"

    def size(self):
        return len(self.items)

    def peek(self):
        if not self.is_empty():
            return self.items[0] # Peek at the front element
        else:
            return "Queue is empty"

# Example Usage
deque_queue = DequeQueue()
deque_queue.enqueue("First")
deque_queue.enqueue("Second")
deque_queue.enqueue("Third")
```

```
print("Is deque queue empty?", deque_queue.is_empty())
print("Size of deque queue:", deque_queue.size())
print("Front element:", deque_queue.peek())

print("\nDequeuing elements:")
print("Dequeued element:", deque_queue.dequeue())
print("Dequeued element:", deque_queue.dequeue())

print("\nSize after dequeues:", deque_queue.size())
print("Is deque queue empty?", deque_queue.is_empty())

print("\nDequeued element:", deque_queue.dequeue()) # Dequeue the last element
print("Is deque queue empty?", deque_queue.is_empty())

print("\nTrying to dequeue from an empty deque queue:", deque_queue.dequeue()) # Trying to dequeue from an empty queue
```

```
Is deque queue empty? False
Size of deque queue: 3
Front element: First

Dequeuing elements:
Dequeued element: First
Dequeued element: Second

Size after dequeues: 1
Is deque queue empty? False

Dequeued element: Third
Is deque queue empty? True

Trying to dequeue from an empty deque queue: Queue is empty
```

GIVE THE CODE FOR TEST OPTIMIZED QUEUE



TASK 3:

Singly Linked List with Traversal

• Task: Implement a Singly Linked List with operations:

insert_at_end(), delete_value(), and traverse().

• Instructions:

o Start with a simple class-based implementation (Node,

LinkedList).

o Use AI to generate inline comments explaining pointer updates

(which are non-trivial).

o Ask AI to suggest test cases to validate all operations.

• Expected Output:

o A functional linked list implementation with clear comments

explaining the logic of insertions and deletions

PROMPT :

WRITE A PYTHON CODE FOR A SINGLYLINKED LIST WITH TWOCLASSES DELETE , TRAVERSE , INSERT

```python
class LinkedList:
    """Represents a singly linked list."""

    def __init__(self):
        """Initializes an empty LinkedList."""
        self.head = None

    def insert_at_beginning(self, data):
        """
        Inserts a new node with the given data at the beginning of the linked list.

        Args:
            data: The data for the new node.
        """
        new_node = Node(data)
        new_node.next = self.head
        self.head = new_node

    def traverse(self):
        """Traverses the linked list and prints the data of each node."""
        current_node = self.head
        while current_node:
            print(current_node.data, end=" -> ")
            current_node = current_node.next
        print("None") # Indicate the end of the list
```

```
# Example Usage for LinkedList
my_list = LinkedList()

# Insert nodes
my_list.insert_at_beginning(30)
my_list.insert_at_beginning(20)
my_list.insert_at_beginning(10)

# Traverse and print the list
print("Linked List:")
my_list.traverse()
```

```
Linked List:
10 -> 20 -> 30 -> None
```

TASK 4:

Binary Search Tree (BST)

• Task: Implement a Binary Search Tree with methods for insert(), search(), and inorder_traversal().

• Instructions:

o Provide AI with a partially written Node and BST class.

o Ask AI to complete missing methods and add docstrings.

o Test with a list of integers and compare outputs of search() for present vs absent elements.

• Expected Output:

o A BST class with clean implementation, meaningful docstrings, and correct traversal output.

PROMPT:

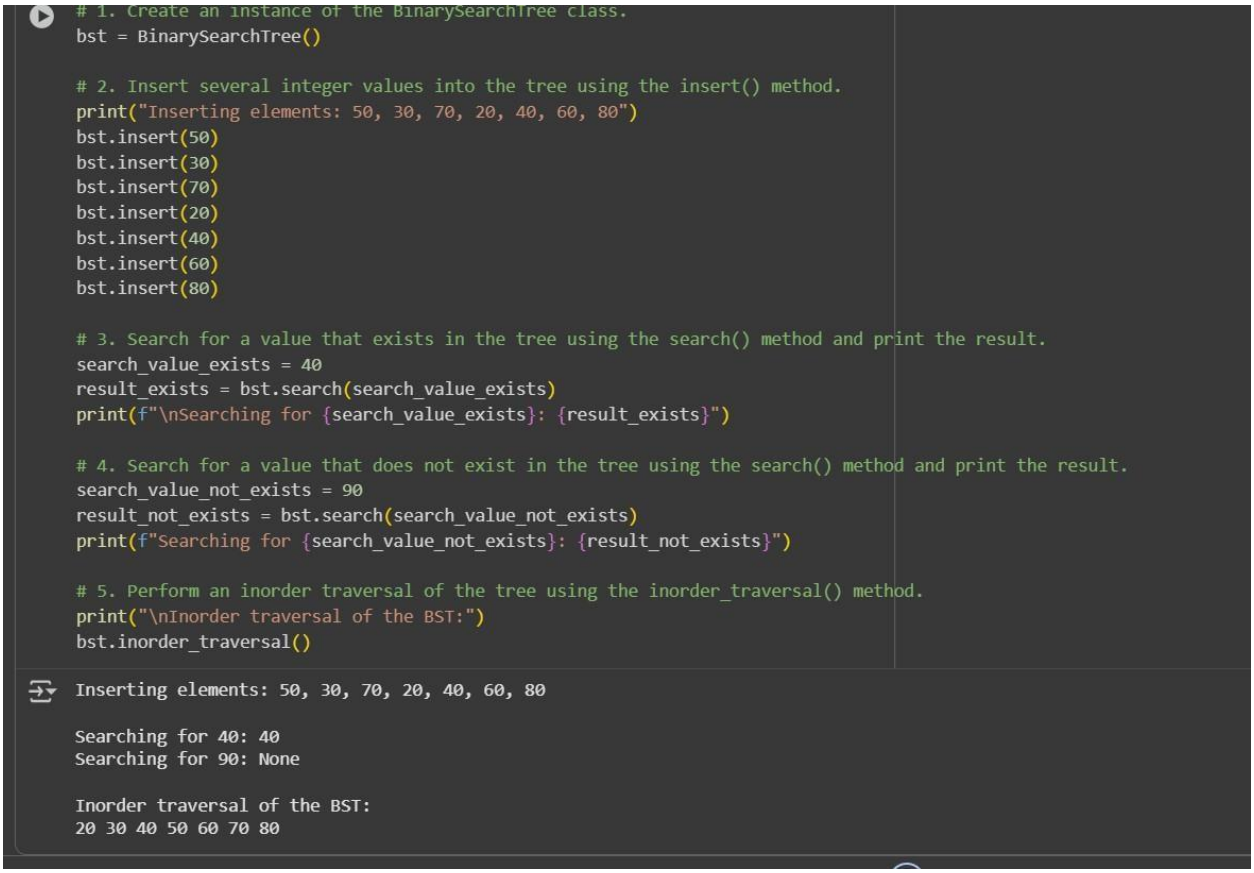
GENERATE A PYTHON CODE FOR BINARY SEARCH TREE WITH METHODS LIKE INSERT() , SEARCH(), INORDER_TRANSVERSAL()

```
class BinarySearchTree:
    """Represents a Binary Search Tree."""

    def __init__(self):
        """Initializes an empty BinarySearchTree."""
        self.root = None

    def insert(self, data):
        """
        Inserts a new node with the given data into the Binary Search Tree.

        Args:
            data: The data to be inserted into the tree.
        """
        if self.root is None:
            self.root = Node(data)
        else:
            self._insert_recursive(self.root, data)

    def _insert_recursive(self, current_node, data):
        """
        Helper function to recursively insert a new node.

        Args:
            current_node: The current node being examined.
            data: The data to be inserted.
        """
        if data < current_node.data:
            if current_node.left is None:
                current_node.left = Node(data)
            else:
                self._insert_recursive(current_node.left, data)
        elif data > current_node.data:
```

```
    def search(self, data):
        """
        Searches for a specific value in the Binary Search Tree.

        Args:
            data: The data to be searched for.

        Returns:
            The data if found, otherwise None.
        """
        return self._search_recursive(self.root, data)

    def _search_recursive(self, current_node, data):
        """
        Helper function to recursively search for a value.

        Args:
            current_node: The current node being examined.
            data: The data to be searched for.

        Returns:
            The data if found, otherwise None.
        """
        if current_node is None:
            return None
        if current_node.data == data:
            return current_node.data
        elif data < current_node.data:
            return self._search_recursive(current_node.left, data)
        else: # data > current_node.data
            return self._search_recursive(current_node.right, data)

    def inorder_traversal(self):
```

```python
def inorder_traversal(self):
    """
    Traverses the tree in inorder (left, root, right) and prints node data.
    """
    self._inorder_recursive(self.root)
    print() # Print a newline after traversal

def _inorder_recursive(self, current_node):
    """
    Helper function for inorder traversal.

    Args:
        current_node: The current node being examined.
    """
    if current_node is None:
        return

    # Traverse left subtree
    self._inorder_recursive(current_node.left)

    # Visit current node
    print(current_node.data, end=" ")

    # Traverse right subtree
    self._inorder_recursive(current_node.right)
```

GIVE THE TEST CODE FOR BINARY SEARCH TREE



```
# 1. Create an instance of the BinarySearchTree class.
bst = BinarySearchTree()

# 2. Insert several integer values into the tree using the insert() method.
print("Inserting elements: 50, 30, 70, 20, 40, 60, 80")
bst.insert(50)
bst.insert(30)
bst.insert(70)
bst.insert(20)
bst.insert(40)
bst.insert(60)
bst.insert(80)

# 3. Search for a value that exists in the tree using the search() method and print the result.
search_value_exists = 40
result_exists = bst.search(search_value_exists)
print(f"\nSearching for {search_value_exists}: {result_exists}")

# 4. Search for a value that does not exist in the tree using the search() method and print the result.
search_value_not_exists = 90
result_not_exists = bst.search(search_value_not_exists)
print(f"Searching for {search_value_not_exists}: {result_not_exists}")

# 5. Perform an inorder traversal of the tree using the inorder_traversal() method.
print("\nInorder traversal of the BST:")
bst.inorder_traversal()
```

```
Inserting elements: 50, 30, 70, 20, 40, 60, 80

Searching for 40: 40
Searching for 90: None

Inorder traversal of the BST:
20 30 40 50 60 70 80
```

TASK 5:

5: Graph Representation and BFS/DFS Traversal

• Task: Implement a Graph using an adjacency list, with traversal methods BFS() and DFS().

• Instructions:

o Start with an adjacency list dictionary.

o Ask AI to generate BFS and DFS implementations with inline comments.

o Compare recursive vs iterative DFS if suggested by AI.

• Expected Output:

o A graph implementation with BFS and DFS traversal methods, with AI-generated comments explaining traversal steps.

PROMPT:

GENERATE A PYTHON CODE TO IMPLEMENT GRAPH USING AN ADJACENCY LIST HAVING TRAVERSAL METHODS LIKE PFS() ,DFS() . ADD INLINE COMMENTS TO EVERY LINE AND ADD FEW TEST CASES

```
# Example Usage
g = Graph() # Create a new Graph object

# Add vertices
g.add_vertex('A')
g.add_vertex('B')
g.add_vertex('C')
g.add_vertex('D')
g.add_vertex('E')

# Add edges
g.add_edge('A', 'B')
g.add_edge('A', 'C')
g.add_edge('B', 'D')
g.add_edge('C', 'E')
g.add_edge('D', 'E')

print("Vertices in the graph:", g.get_vertices()) # Get and print all vertices
print("Neighbors of A:", g.get_neighbors('A')) # Get and print neighbors of vertex A
print("Neighbors of B:", g.get_neighbors('B')) # Get and print neighbors of vertex B
print("Neighbors of F:", g.get_neighbors('F')) # Try to get neighbors of a non-existent vertex
```

```
def get_vertices(self):
    """
    Returns a list of all vertices in the graph.
    """
    return list(self.graph.keys()) # Return the keys of the adjacency list

def get_neighbors(self, vertex):
    """
    Returns a list of neighbors for a given vertex.

    Args:
        vertex: The vertex to get neighbors for.

    Returns:
        list: A list of neighboring vertices, or an empty list if the vertex doesn't exist.
    """
    return self.graph.get(vertex, []) # Return the list of neighbors for the vertex, or an empty list if vertex not
```

```python
class Graph:
    """Represents a graph using an adjacency list."""
    def __init__(self):
        # Adjacency list is a dictionary where keys are vertices and values are lists of adjacent vertices
        self.graph = {}

    def add_vertex(self, vertex):
        """
        Adds a vertex to the graph if it doesn't already exist.

        Args:
            vertex: The vertex to add.
        """
        if vertex not in self.graph:
            self.graph[vertex] = [] # Add the vertex as a key with an empty list for its neighbors

    def add_edge(self, u, v):
        """
        Adds an edge between two vertices u and v.
        Assumes an undirected graph, so adds edges in both directions.

        Args:
            u: The first vertex.
            v: The second vertex.
        """
        # Ensure both vertices exist in the graph
        self.add_vertex(u)
        self.add_vertex(v)

        # Add edge from u to v if not already present
        if v not in self.graph[u]:
            self.graph[u].append(v)

        # Add edge from v to u if not already present (for undirected graph)
        if u not in self.graph[v]:
            self.graph[v].append(u)
```

```python
# Create an instance of the Graph class
graph = Graph()

# Add edges to the graph
graph.add_edge('A', 'B')
graph.add_edge('A', 'C')
graph.add_edge('B', 'D')
graph.add_edge('C', 'E')
graph.add_edge('D', 'E')
graph.add_edge('D', 'F')
graph.add_edge('E', 'F')

# Demonstrate BFS traversal
graph.bfs('A')

# Demonstrate DFS traversal
graph.dfs('A')
```

```
BFS traversal starting from A:
A B C D E F
DFS traversal starting from A:
A B D E C F
```

Summary